

Compiler Design Laboratory

Experiment 2: Advanced Lexical Analysis Using Flex

Objective

To design a more complete lexical analyzer using Flex and to understand the use of different regular expression notations and token classification.

1. Revision: Compiling and Running a Flex Program

1. Write the Flex program and save it as `lexer.l`
2. Generate C file:

```
flex lexer.l
```

3. Compile the generated file:

```
gcc lex.yy.c -o lexer -lfl
```

4. Run the program:

```
./lexer < input.c
```

2. Regular Expression Notations in Flex

Flex supports the following regular expression notations:

2.1 Basic Symbols

Notation	Meaning
<code>a</code>	Matches the character 'a'
<code>.</code>	Matches any character except newline
<code>[abc]</code>	Matches any one of a, b, or c
<code>[a-z]</code>	Matches any character from a to z
<code>[0-9]</code>	Matches any digit

2.2 Repetition Operators

Notation	Meaning
<code>*</code>	Zero or more times
<code>+</code>	One or more times
<code>?</code>	Zero or one time
<code>{m,n}</code>	Between m and n times

2.3 Grouping and Alternation

Notation	Meaning
(r)	Grouping
r s	r or s
rs	Concatenation
^r	Beginning of line
r\$	End of line

3. Sample Advanced Flex Program

The following Flex program recognizes keywords, identifiers, numbers, operators, and separators and prints tokens.

```
%{
#include <stdio.h>
int line_no = 1;
%}

%%
"int"|"if"|"else"|"while"|"return"    { printf("<KEYWORD,_%s>\n",
    yytext); }

[a-zA-Z_][a-zA-Z0-9_]*                  { printf("<ID,_%s>\n",
    yytext); }

[0-9]+                                  { printf("<NUM,_%s>\n",
    yytext); }

"=="|"!="|"<="|">="|"<"|">"          { printf("<RELOP,_%s>\n",
    yytext); }

"+"|"-"|"*"|"/"|"="                  { printf("<OP,_%s>\n",
    yytext); }

";"|","|"("|")"|"{"|"}"              { printf("<SEP,_%s>\n",
    yytext); }

\n                                      { line_no++; }

[ \t]+                                  ;    /* Ignore spaces and tabs */

.                                       { printf("<INVALID,_%s>_at_
    line_%d\n", yytext, line_no); }
%%
```

```
int main() {  
    yylex();  
    return 0;  
}
```

4. Assignment for Students

Task 1

Extend the above Flex program to:

- Count number of keywords, identifiers, numbers, operators, and separators
- Print the total count at the end

Task 2

Modify the lexer to:

- Maintain a symbol table for identifiers
- Print the symbol table at the end

Task 3

Add support for:

- Single-line comments (//)
- Multi-line comments (/* ... */)

Task 4

Modify the program to:

- Report lexical errors with line numbers
- Continue scanning even after errors

Learning Outcomes

After completing this experiment, students will be able to:

- Write a complete lexical analyzer using Flex
- Use different regular expression operators
- Recognize and classify tokens
- Handle errors and maintain symbol table

5. Test Cases and Expected Output

This section provides sample input programs and their expected outputs to help students verify the correctness of their lexical analyzer.

5.1 Test Case 1: Basic C Statements

Input File: test1.c

```
int a = 10;
int b = 20;
if (a < b) return a;
```

Expected Output:

```
<KEYWORD, int>
<ID, a>
<OP, =>
<NUM, 10>
<SEP, ;>
<KEYWORD, int>
<ID, b>
<OP, =>
<NUM, 20>
<SEP, ;>
<KEYWORD, if>
<SEP, (>
<ID, a>
<RELOP, <>
<ID, b>
<SEP, )>
<KEYWORD, return>
<ID, a>
<SEP, ;>
```

5.2 Test Case 2: Operators and Expressions

Input File: test2.c

```
sum = a + b * 25;
while (sum >= 100) sum = sum - 1;
```

Expected Output:

```
<ID, sum>
<OP, =>
<ID, a>
<OP, +>
<ID, b>
```

```
<OP, *>
<NUM, 25>
<SEP, ;>
<KEYWORD, while>
<SEP, (>
<ID, sum>
<RELOP, >=>
<NUM, 100>
<SEP, )>
<ID, sum>
<OP, =>
<ID, sum>
<OP, ->
<NUM, 1>
<SEP, ;>
```

5.3 Test Case 3: Invalid Tokens and Error Reporting

Input File: test3.c

```
int x = 10;
y = x @ 5;
```

Expected Output:

```
<KEYWORD, int>
<ID, x>
<OP, =>
<NUM, 10>
<SEP, ;>
<ID, y>
<OP, =>
<ID, x>
<INVALID, @> at line 2
<NUM, 5>
<SEP, ;>
```

5.4 Test Case 4: Whitespace and Newlines

Input File: test4.c

```
int      a      =      5;

return      a;
```

Expected Output:

```
<KEYWORD, int>
```

```
<ID, a>
<OP, =>
<NUM, 5>
<SEP, ;>
<KEYWORD, return>
<ID, a>
<SEP, ;>
```

6. Student Verification Instructions

- Run the lexer on each test file:

```
./lexer < test1.c
./lexer < test2.c
./lexer < test3.c
./lexer < test4.c
```

- Compare the output with the expected output given above.
- If there is any mismatch, debug and correct the Flex program.

Additional Requirement: Symbol Table Creation and Maintenance

In this experiment, each student must implement a **Symbol Table** as part of the lexical analyzer.

What to Store in the Symbol Table

The symbol table should store information about **identifiers** encountered in the input program. Each entry in the symbol table should contain at least the following fields:

- Name of the identifier
- Data type (if available or marked as unknown)
- Line number of first occurrence

When to Insert into the Symbol Table

- Insert an identifier into the symbol table **only if it is not already present**.
- If the identifier already exists, do not create a duplicate entry.

How to Use the Symbol Table

- Every time an identifier token is recognized, the lexical analyzer must:
 - Check whether it is already present in the symbol table.
 - If not present, insert it.
- Keywords must **not** be stored in the symbol table.

Printing the Symbol Table

At the end of the lexical analysis:

- Print the symbol table in a tabular format with proper headings.
- The table should show all unique identifiers found in the input program.

Example Output Format

Symbol Table		

Name	Type	Line No

a	int	1
b	int	2
sum	int	4

Important Notes

- Only identifiers should be stored in the symbol table.
- Do not store keywords, operators, or constants.
- The symbol table should not contain duplicate entries.

How to Implement the Symbol Table in Code

In this experiment, the symbol table must be implemented using a dynamic data structure in C.

Data Structure for Symbol Table

Each entry of the symbol table is represented using a structure:

```
struct Symbol {
    char name[50];
    int line_no;
};

struct SymbolTable {
    struct Symbol *table;
    int size;
    int capacity;
};
```

Initialize the Symbol Table

```
void initSymbolTable(struct SymbolTable *st) {
    st->size = 0;
    st->capacity = 10;
    st->table = (struct Symbol *)malloc(st->capacity * sizeof(
        struct Symbol));
}
```

Lookup Function

This function checks whether an identifier is already present in the symbol table.

```
int lookup(struct SymbolTable *st, const char *name) {
    for (int i = 0; i < st->size; i++) {
        if (strcmp(st->table[i].name, name) == 0)
            return i;    /* Found */
    }
    return -1;    /* Not found */
}
```

Insert Function

This function inserts a new identifier only if it is not already present.

```
void insert(struct SymbolTable *st, const char *name, int line_no) {
    if (lookup(st, name) != -1)
        return;    /* Already exists */

    if (st->size == st->capacity) {
        st->capacity *= 2;
```



```

        st->table = (struct Symbol *)realloc(st->table,
                                             st->capacity * sizeof(struct Symbol));
    }

    strcpy(st->table[st->size].name, name);
    st->table[st->size].line_no = line_no;
    st->size++;
}

```

Print Function

```

void printSymbolTable(struct SymbolTable *st) {
    printf("\n-----\n");
    printf("Symbol Table\n");
    printf("-----\n");
    printf("Name\tLine No\n");
    printf("-----\n");

    for (int i = 0; i < st->size; i++) {
        printf("%s\t%d\n", st->table[i].name, st->table[i].
              line_no);
    }
    printf("-----\n");
}

```

How to Use Symbol Table in Flex Program

Declare the symbol table in the header section of the Flex file:

```

%{
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int line_no = 1;
struct SymbolTable symtab;
%}

```

Initialize it in main():

```

int main() {
    initSymbolTable(&symtab);
    yylex();
    printSymbolTable(&symtab);
    return 0;
}

```

Insert Identifier from Lex Rule

Modify the identifier rule as follows:

```
[a-zA-Z_][a-zA-Z0-9_]* {  
    printf("<ID, \u005Cs>\n", yytext);  
    insert(&syntab, yytext, line_no);  
}
```

Line Number Tracking Rule

```
\n    { line_no++; }
```

Important Notes

- Only identifiers should be inserted into the symbol table.
- Keywords must NOT be inserted.
- Duplicate entries must be avoided using the lookup function.
- The symbol table must be printed at the end of execution.

— End of Experiment —